LEO: Line End Optimizer for Sub-7nm Technology Nodes

Diwesh Pandey
diweshpandey@in.ibm.com
IBM Systems
Bengaluru, Karnataka, India

Gustavo E. Tellez
tellez@us.ibm.com
IBM T.J. Watson Research Center
Yorktown Heights, NY, USA

James Leland
leland@us.ibm.com
IBM Systems
Poughkeepsie, NY, USA

ABSTRACT

Sub-7nm technology nodes have introduced new challenges, specifically in the lower metal layers. Extreme Ultraviolet Lithography (EUV) and multi-patterning-based lithography such as Self-Aligned Double Patterning (SADP) solutions have become key choices for the manufacturing of these layers. The demand for microprocessors has increased tremendously in the last few years and this imposes another challenge to the chip manufacturers to build their products at a very rapid rate. These days a mix of different lithography solutions for the manufacturing of metal layers is quite common. We propose a first-of-its-kind routing plugin which solves design rule violations for multiple lithography technologies without making any changes in the existing routers. Our plugin consists of a practical line-end optimization (LEO) algorithm, which solves most line-end problems in a few minutes, even for very large designs. Our solution is implemented in the development of a 7nm, industrial microprocessor design.

CCS CONCEPTS

• **Hardware → Electronic design automation**;

KEYWORDS

extreme-ultraviolet lithography, self-align double patterning, cut mask optimization, detail routing

ACM Reference Format:
Diwesh Pandey, Gustavo E. Tellez, and James Leland. 2022. LEO: Line End Optimizer for Sub-7nm Technology Nodes . In *Proceedings of the 2022 International Symposium on Physical Design (ISPD '22), March 27–30, 2022, Virtual Event, Canada.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3505170.3506726

1 INTRODUCTION

The lower metal layers in the back end of line (BEOL) stack are most impacted by technology shrink and increasing design complexities. In the scenario of rapid technology changes where mixed lithographic solutions are not very uncommon, it becomes essential to develop routing tools to cope with various foundry imposed specific rules. The impact of these design rules is high in clock routing and signal routing tools. Whereas power routing is usually done in very specific patterns of metal wires, therefore the impact due to complex foundry rules is less. Some of the design rules in sub 7nm technology nodes are very complex to model and demand disruptive changes in the signal routing engine. these nontrivial design rules require new algorithms and a huge amount of time in their implementation. This long implementation time impacts adversely the chip delivery timeline. Similarly, the required changes in the clock router further pushes the chip delivery timeline. In summary, the frequent change in the technology or using a mix of different technologies enhances the algorithmic complexities and requires many changes in the design flow.

Any algorithmic enhancement inside a detail router heavily depends on the underlying infrastructure and the net data structure. There is no easy way to port these new algorithms which are targeted to solve the new technology-driven rules, to a different router. This limits the flexibility in collaboration and to easily port the new detail routing utilities across the industry and academia. Therefore it becomes essential to develop a routing tool which can cope with the rapid change in the technology and without having any dependency on the routing tools and the underlying infrastructure.

We propose a routing plugin which is developed independent of the existing routers (signal and clock). Our solution is built upon a metal shape processing engine, which can be run on top of any academic or industry routers to solve a few complex design rules. If any new foundry-specific rule gets added in the future, the plugin can be easily extended and quickly tested, without the need of qualifying the whole tool-chain again. In this paper, we model design rules from two patterning technologies SADP and EUV, and will provide a solution for both of them. We will present a line-end optimization tool independent of the detail router. In our paper, we will use the term "detail router", for both signal and clock router.

The first patterning technique, SADP, consists of on-track shapes [1][2] and cut masks containing trim shapes (or trims) for each layer are applied to fabricate 1D wire patterns. Along a track, a trim shape determines where metal shapes begin and end. The beginning/end of a metal shape is called a line end, or a tip. In SADP, trims are used to remove the metal shapes, such that gaps are formed. As a result, these gaps result in disconnected metal segments. The routing wires are active shapes in metal tracks, while the portion of a metal track between two trims which is not assigned to any wire, is inactive or dummy metal. In Figure 1(a), uniformly aligned metal shapes are shown and represent metal track patterns in a horizontal layer. In Figure 1(b), trims are superimposed on these regular continuous shapes to create a discontinuity in tracks and to form the actual wire pattern used in routing. Eventually, the wires form the layout as shown in Figure 1(c). For simplification, the inactive metal in tracks will be omitted (shown as part of the gap) in all our diagrams.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ISPD '22, March 27–30, 2022, Virtual Event, Canada
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9210-5/22/03…$15.00
https://doi.org/10.1145/3505170.3506726

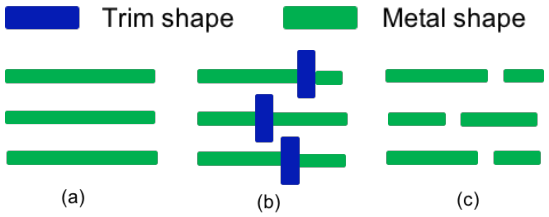


Figure 1: Wire fabrication using trim shapes.(a) Core mask pattern (b) trim shapes on core mask (c) final layout.

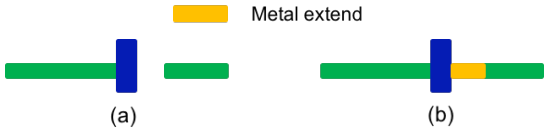


Figure 2: trim and extent generation. (a) Trim shape position in metal gap (b) Extent connected to trim.

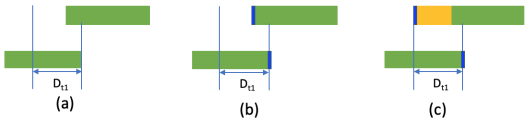


Figure 3: EUV: Line end modelling (a) Line end spacing between two metal shapes (b) Line end position represented by trims (c) Extent connected to line end trim

The position of the SADP trim shapes determines the location of the line end edges. Due to design rules, the position of a trim shape is constrained by the position of trim shapes on neighboring tracks. Consequently, the position of any line end edge becomes constrained by the position of any line end edge in a sufficiently close, neighboring track.

The second patterning technique, EUV, uses a single exposure for the direct printing of the metal layers. However, due to a very small tip to tip spacing, pattern fidelity, and pattern variability become challenging. To cope with these challenges, EUV brings a variety of new line end spacing ground rules and these rules are apparently similar to the SADP trim rules, in that they both constrain line end positions in neighboring tracks.

In a typical routing flow, trims are inserted at the line end positions of the wire shapes. To place a trim shape legally, the size of the gap between the line ends of two wires must be greater than or equal to the minimum trim width. Also, a larger gap provides flexibility for the movement of trim. This important feature is key, as the movement of trim along the gaps can immensely influence routing behavior. Note: Dummy metal present is subject to a metal minimum area rule. Once a trim takes its final position (as shown in Figure 2(a)) in the gap, the wiring shape needs to be extended to the trim and this is achieved by using an "extent" metal shape, as illustrated in Figure 2(b).

In EUV, due to the direct printing of the metal shapes, there is no requirement for a trim mask. Still, EUV has complex line end spacing constraints to reduce process variation. Similar to SADP technology, we use "extent" metal shapes to meet the line end constraints, making the line end extent optimization problem similar to SADP trim and extent optimization. Figure 3(a), depicts line end spacing between two metal line ends, which requires a minimum spacing of D_{t1}. Similar to SADP technology, the metal line end positions in EUV can be modeled by trim shapes as shown in Figure 3(b). But the trim shapes in EUV have no significance, hence these shapes can be assumed as artificial trims and are just used for modeling the problem to share a common infrastructure. Extent metal is used to meet the minimum spacing of D_{t1} between two metal line ends, as shown in Figure3(c).

To the best of our knowledge, no research work has focused on developing a stand alone plugin outside of the detail router and formulation for solving EUV driven line end rules. Additionally, there are no prior arts which present a study of the line end spacing constraints and their optimization in EUV. Though there have been some existing studies considering complicated line end rules in SADP technology. The work in [3][4][5] is primarily focused on solving layout decomposition problem. They have studied the cut mask rules in detail, but none of the work is aligned towards the study of impact of cut mask rules in detail routing.

Line-end extension techniques have been used to optimize the cut mask manufacturability in [6][2]. Zhang et al [6] proposed a greedy algorithm to extend the line ends of metal wires to minimize the gaps in the layout. The proposed algorithm is used in the design of standard cells. Though it deploys the mechanism of adding extent wires in the gap, but the idea is only limited to cell design space. It can't be easily generalized in the framework of detail routing, where the underlying infrastructure is much complex. Zhang et al [2] proposed a line-end extension technique to improve the layout manufacturability. This work is primarily focused on mask cost reduction techniques, whereas we focus on a new plugin based detail routing infrastructure to achieve faster routing tool development cycle for a new technology, using a standalone tool framework. In [7] impact of SADP routing on standard cell pin access is assessed, and a solution is proposed to optimize the local pin access using line end extension technique. It studies the SADP specific constraints in detail and formulates and MILP based technique to maximize pin access points. But this work is also limited to the pin access optimization and can not be generalized to detail routing.

[8], Fang proposed a cut mask optimization technique along with wire planning in a SAMP(self align multiple patterning) aware detail router. It classifies the cut mask violations into three categories in terms of violation directions relative to wires, and then the corresponding routing rules are identified. It shows that to generate cut mask-friendly full-chip routing results, considering SAMP only in the detailed routing stage is not effective. Thus, it proposes a cut mask-aware wire planning during track routing with integer linear programming (ILP)-based approach. Additionally, cut mask-aware detailed routing and post-layout modification techniques are proposed for further cut mask optimization. The result shows that, with a SAMP-aware full-chip router, in comparison to the baseline router, the mask violations can be further reduced by about 66% on average.

However, this work is focused only on SAMP related cut mask rules, and many complicated design rules related to EUV technology are completely different and therefore require different modeling.

This implementation is tightly integrated within track routing and detail routing hence it requires significant changes in these tools. If any new foundry specific rule gets added in the future, it requires to tune and qualify the whole toolchain again. This approach takes roughly 10 minutes to run on an average size design, and it may not be a viable solution for very large industry designs with more than a million nets.

Recently, Ying-Chi Wei et al. [9] proposed a detail router tackling with dynamic line end spacing constraints related to cut spacing design rules for the cut masks. It creates a line end map for fast identification of spacing violation in detail routing kernel. A routing cost formulation is proposed along with a net ordering scheme. The result shows that it can solve all end-end spacing violations as well as 75% of parity spacing violations with a runtime increase of 14%. However, this work is also only focused on cut spacing design rules for the cut masks. Additionally, it is integrated inside a detail router and this approach is not viable when dealing with line end rules associated with multiple lithography technologies. Any change or new addition in the rule will require an update in the line-end map, cost parameters and, re-tuning of the detail router. This technique solves 75% of parity violations and parity spacing is the hardest rule considered in this work. However, this rule is just one subset of all of the adjacent line end spacing rules we have outlined in our paper.

In this paper, we present a novel routing plugin based optimization approach, for simultaneously addressing line end optimization in both EUV and SADP technology. Our results will show that a majority of the line end spacing violations can be solved outside of the detail router. Additionally, we also formulate an interaction between the line end optimization solver and the detail router.

1.1 Our Contributions

We summarize our main contributions below:

(1) Introduction of line end spacing rules in EUV.
(2) The routing plugin infrastructure for modeling of line ends in both SADP and EUV technology using constraint graph and ILP formulation.
(3) A graph pruning mechanism that enables parallelism by creating multiple subgraphs from a constraint graph.
(4) Set of algorithms:
  (a) The longest path-based algorithm to solve subgraphs.
  (b) A min cost-based extent length optimization algorithm.
(5) Solution for routing based line end optimization problem.
(6) Routing plugin based novel design methodology flow.

The remainder of this paper is organized as follows: Section II shows the introduction of line end spacing rules in EUV and trim rules in SADP, and a mathematical formulation of constraint graph to model the trims in both SADP and EUV. Section III explains the problem formulation using MILP. Section IV introduces the key algorithms to solve the constraint graph and a routing based line end optimization flow. Experimental results are presented in section VI and section VII concludes the paper.

2 PRELIMINARIES

In this section, under section II A and II B, we introduce line end specific rules in EUV and trim specific rules in SADP respectively.

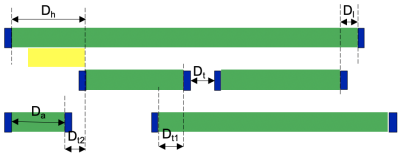


Figure 4: EUV line end spacing constraints. Same track line end spacing $\geq D_t$, min length $\geq D_a$, adjacent track opposing direction spacing $\geq$ ($\mathbf{D}_{t1}$ or D_{t2}), Adjacent track same direction spacing ($\leq D_l$, or $\geq \mathbf{D}_h$)

2.1 Line end spacing rules formulation in EUV

To enable EUV friendly layout, we formulate the EUV driven line end spacing constraints into specific design rules. We derive the following four design rules as shown in Figure 4.

Same track line end spacing constraint: Two metal line ends in the same track should have a minimum distance D_t between them.

Minimum length constraints: Two metal line ends in the same track should have a minimum distance D_a, derived from a minimum length rule or a minimum area rule.

Diff track opposing direction line end spacing constraint: When two line ends in different tracks are in opposing direction to each other, they should be a minimum distance (D_{t1} or D_{t2}) apart.

Diff track same direction line end spacing constraint: When two metal line ends are in the same direction, the two line-end positions should never be inside a forbidden region as shown in a yellow rectangle in Figure 4. The spacing between the two line ends should be less than D_l or greater than D_h.

2.2 Trim routing rules in SADP

Cut mask rules for routing have been discussed in detail in paper [8], which describes the width and spacing rules for cuts (trims) in the cut mask. However, in our problem formulation, the trim shapes are modeled in gaps between tracks, which require a new set of rules for trim-trim interaction and trim-wire interaction. In the paper, [10] trim rules are modeled for poly conductor "PC" layer and not generalized to routing layers. We have briefly illustrated trim rules in Figure 5.

Trim width: All trim shapes should have a constant width W.

Same track trim spacing: This is minimum spacing (D_t) between two trims placed at the line ends of the metal shapes. Due to the dummy metal shape in the gap, the min spacing rule should also be compliant with the metal minimum area and minimum length rule.

Adjacent track trim spacing: When two trims are in adjacent tracks, there should be a minimum distance (D_u) between them.

Minimum trim height: Trim must meet a minimum height (h) criteria from technology.

Trim edge-edge distance: When trims are aligned, they must meet a minimum distance (d) rule or they must be merged to a single trim.

3 PROBLEM FORMULATION

We derive technology-independent modeling of trim and extents using a constraint graph $G(V, E)$. We are given a layout $L(R)$, which consists of rectangular metal shapes $R = \{R_i | i = 1, \ldots, |R|\}$. This

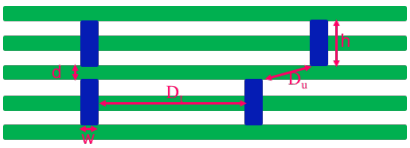


Figure 5: SADP trim rules. Trim min width =W, same track trim spacing $\geq D_t$, diff track trim spacing $\geq D_u$, trim height $\geq$ h, min trim distance $\geq$ d

layout is the result of a routing of a design. The metal shapes are assumed to be minimal: that is they cannot be any smaller without either disconnecting the layout or introducing spacing rule violations with other shapes, for example via cut shapes. For SADP, we model a potential trim shape S_i as a node $v_i \in V$ in the graph $G(V, E)$. For each trim shape S_i, a variable x_i represents the position of the trim shape. The position of each trim shape S_i is bounded by the positions of the rectangles: the lower bound is denoted L_i, the upper bound is denoted U_i. For EUV, we model the position of a line end, or tip T_i as a node $v_i \in V$ in the graph $G(V, E)$. Each rectangle has two movable line ends. The variable x_i represents the position of the line end. As with a trim shape, line end positions are bounded by the minimal shapes of their rectangles.

The arcs $e_{i,j} \in E$ in $G(V, E)$, represent spacing relationships between neighboring trim shapes S_i and S_j, or between neighboring tips T_i and T_j. Each arc is accompanied by a set of constants, $\{W_{i,j}, D^h_{i,j}, D^l_{i,j}\}$, and a set of integer variables, $\{a_{i,j}, b_{i,j}, c_{i,j}\}$. Values, $D^h_{i,j}$, $D^l_{i,j}$ are different spacing constraints between nodes. The spacing relationships between two trim shapes are then given by the following equations:

$$x_j - x_i - a_{i,j} * D^l_{i,j} + b_{i,j} * W_{i,j} \geq 0 \quad (1)$$

$$x_i - x_j + a_{i,j} * W_{i,j} - b_{i,j} * D^h_{i,j} \geq 0 \quad (2)$$

$$c_{i,j} <= a_{i,j} + b_{i,j} <= 1, a_{i,j}, b_{i,j}, c_{i,j} \in \{0, 1\} \quad (3)$$

The purpose of the integer values $a_{i,j}$, $b_{i,j}$ and $c_{i,j}$ is to implement the "or" constraints, which occur due to disjoint intervals of the spacing constraints. We select the value of the constant $W_{i,j} >> U_j - L_i$, such that when the associated integer variable is 1, the constraint becomes effectively unbounded. Our optimization objective is to minimize the length of the line end extents. To model this objective we assign a weight to each node n_i, $w_i \in \{-1, 0, 1\}$, which drives the trim shape towards its optimal solution. To note here that an individual trim shape's ideal solution is trivially $x_i = L_i (w_i = 1)$ or $x_i = U_i (w_i = -1)$.

We now formulate the **Line End Optimization (LEO) Problem**: given a graph $G(V, E)$:

$$\begin{array}{ll} \textbf{minimize} & \sum_{i \in n,} w_i x_i \\ \textbf{subject to:} & (1), (2), (3) \end{array}$$

$$L_i <= x_i <= U_i \quad (4)$$

The problem as formulated is a MILP (*Mixed Integer Linear Program*). Given a selection of values of the integer variables, the resulting problem is a special linear programming problem known as a *network linear programming problem*. We will use this fact to later to develop our solution. Furthermore, since network linear programming problems are *unimodular*, provided that all the constraint

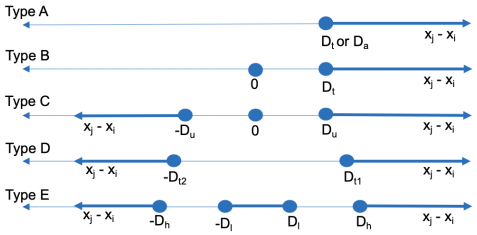


Figure 6: Modelling the feasible regions of arcs and their spacing constraints as inequality intervals.

constants are integers, then the solution will consist of integers, which is an important requirement in real designs.

The SADP and EUV design rules are modeled with the following simplifications of the constraints $(1), \ldots, (3)$:

(1) **Type "A"** arcs: set $a_{i,j} = 1, b_{i,j} = 0$, and $D^l_{i,j} = D_t$ or $D^l_{i,j} = D_a$.
(2) **Type "B"** arcs: set $b_{i,j} = 0$, and $D^l_{i,j} = D_t$.
(3) **Type "C"** arcs: set $c_{i,j} = 0$, $D^l_{i,j} = D_u$, $D^h_{i,j} = D_u$.
(4) **Type "D"** arcs: set $c_{i,j} = 1$, $D^l_{i,j} = D_{t_1}$ and $D^h_{i,j} = D_{t_2}$.
(5) **Type "E"** arcs: set $c_{i,j} = 1$, $D^l_{i,j} = D_h$ and $D^h_{i,j} = D_h$, set $c_{i,j} = 1$, $D^l_{i,j} = -D_l$ and $D^h_{i,j} = -D_l$.

For each arc type, the spacing inequalities may be viewed as inequality intervals as shown in Figure 6. The arc constraints define intervals of feasible distances between the variables and are applied depending on the constraints implied by the technology's rules.

(1) **Type "A"** arcs are used for simple rules like a minimum area metal rule or a minimum spacing rule on the same track.
(2) **Type "B"** arcs allow two trim shapes on the same track to either merge or stay apart by a minimum spacing D_t.
(3) **Type "C"** arcs allow two trim shapes on adjacent tracks to either merge or to stay apart by a minimum spacing D_u.
(4) **Type "D"** arcs model the adjacent track, same direction tip to tip EUV spacing, and
(5) **Type "E"** arcs model the adjacent track, opposite direction tip to tip EUV spacing.

A solution to the **LEO** problem may not exist. That is, in practice, the routed shapes may be placed in such a way that no feasible line end extension solution is possible. Hence, we formulate a second problem, the **Routing-based line end optimization (rLEO)** problem. In this problem, we would like a path search solution that leaves enough space for the **LEO** problem to be solved. During path search, regions at ends of shapes can be considered blocked to path ends, in order to satisfy the line end constraints.

4 SOLUTION

In this section, we outline the solution to the **rLEO** problem. The solution consists of several steps:

- A detail routing solution which is not line-end aware.
- A partitioning of the problem into sub-problems.
- A solution to the **LEO** problem, which also returns unsolved sub-problems.
- Rip-up and reroute of the wires and vias involved in the unsolved sub-problem.
- Iterative application of the last two steps

The constraint graph can become very large for a large design, hence methods are required to partition the graph for better efficiency and to enable parallelism. In this section, we will describe a method to partition the constraint graph in multiple independent subgraphs. We will then show how to solve efficiently the **LEO** problem in the sub-graphs, and finally, we will outline the full solution to the **rLEO** problem, wherein we integrate the router and solvers.

4.1 Graph partitioning

The constraint graph, created in section III, is being built assuming that a node might travel all along a gap or between metal line end positions. This makes the graph very large, as nodes situated at a very long distance are also connected and these nodes do not influence each other's legal position. Hence, to add a pruning heuristic:

- The position x_i of each node n_i has a feasible range bounded by (4), $[L_i, U_i]$. The size of this range is $S_i = U_i - L_i$.
- We restrict the distance a trim will travel by a *prune distance* P, such that $S_i >= P = k * D_t$. Here k is a called *prune factor* and is a positive integer.
- The prune distance effectively reduces the size of the gap for a trim shape. Hence the feasible range for a node n_i becomes $[L_i, L_i + P]$ or $[U_i - P, U_i]$.
- The pruned interval is chosen based on the ideal location of the node. We denote the pruned feasible range as $[L_i^P, U_i^P]$.
- Given an arc $a_{i,j}$, if $L_j^P - U_i^P > D_{ij}$ then the positions of the arc's nodes will always be far enough apart that the constraint of the arc will always be satisfied. Thus, this arc may be removed from the graph. In general, if the feasible regions of two nodes do not intersect then no arcs are needed between those nodes.

A smaller prune value imposes more restriction and allows the pruning of more arcs. Consequently, the graph becomes extremely sparse and disconnected. We take advantage of this fact and collect the resulting connected components of the graph into *sub-graphs*. The number of sub-graphs increases as we restrict the movement of the nodes using the prune factor. Each of the sub-graphs can be solved independently and in parallel. Restricted node movement also increases the chance of an infeasible problem, and degrades the objective of the solution. Hence it is important to choose an appropriate value for k. The assessment of prune factor values is done in Table 3 in section V.

4.2 Algorithms

We propose solution for the **LEO** problem, with a set of algorithms to target both run time and quality. The overall solution consists of:

(1) Longest path-based algorithm on subgraphs.
(2) Extent minimization algorithm on subgraphs.

4.2.1 Heuristic integer variable assignments (LPA solver). We derive a linear program from the **LEO** MILP by assigning some integer constraints and relaxing the remaining integer constraints. Without integer constraints, the **LEO** is a simpler linear program. The feasible solution for this simpler problem may be found using the Longest Path algorithm, so-called *LPA* (primal), or a min-cut algorithm (dual). We use the *LPA* in our solution, which in practice results in a very fast run time. The **LEO** algorithm explores the space of integer variable assignments heuristically. The algorithm uses the following *LPA* based oracle to determine whether a particular sub-problem is feasible, as follows:

(1) Using LPA, all the nodes are placed at their minimal feasible positions, denoted x_i^L, such as shown in Figure 7.
(2) Using LPA, all the nodes are placed at their maximal feasible positions, denoted x_i^U.
(3) The slack of an arc can be computed as $K_{i,j} = x_j^U - x_i^L - D_{i,j}$.
(4) An infeasible problem is detected by the LPA, when $K_{i,j} < 0$.

The **LEO** algorithm is a heuristic depth-first traversal of the integer variable decision tree. The **LEO** algorithm steps through the arcs of types A through E in a specific order, and for each arc determines whether a feasible solution exists, using the LPA oracle on each of the arcs feasible regions. For a given arc, the feasible regions are also ordered. The first region that produces a feasible partial solution is used, and no backtracking is done. When a feasible region is found for an arc, the arc is permanently added to the graph, and the next arc in the order is visited. If no feasible region is found for an arc, the arc is discarded, resulting in a design rule violation (DRV). The algorithm terminates after all arcs are visited, and a final feasible solution is found. Key to the **LEO** algorithm is the heuristic ordering of the arcs, and for each arc of the feasible regions. We use the following ordering heuristics:

(1) Group arcs, by type, in alphabetical order. The groups are therefore ordered number of feasible regions in the arc. Hence, simpler arcs are processed first.
(2) Order the arcs within each group by their initial slack $K_{i,j}$. The initial slack is obtained from the LPA oracle before the algorithm starts. Hence tighter arcs within a group, are processed first.
(3) Order the feasible regions for each arc from left to right (or right to left) such that the region nearest to the ideal solution is tried first.

Figure 9 depicts a feasible solutions with the final position of nodes after running LPA. Note that a poor selection of integer variables results in an infeasible Longest Path problem and we navigate the infeasible problems by trying multiple values for a given variable selection, within the order that we have described. Hence, in the end, we only try to solve a linear number of Longest Path problems. Our graph is a directed cyclic graph with positive and negative arcs, where an arc $a_{i,j}$ is positive if $j > i$, otherwise, it's negative. The LPA solver has polynomial runtime complexity. Let I be the number of feasible ranges on the integer variables, then $I = O(|E|)$. The runtime of the LPA is $O(|E|^2)$, hence the LPA based **LEO** algorithm complexity is $O(|E|^3)$. In practice, the number of cycles in the constraint graphs is small thus the runtime of the algorithm is $O(|E|^2)$. Since the graphs are very sparse, the practical complexity is then $O(|V|^2)$.

4.2.2 Extent minimization algorithm(EMA). The LPA solver results in a feasible solution. However, the extent length of this solution can be poor. To achieve improvement in extent length, we implement an algorithm inspired by the *simplex* algorithm, on the graph resulting from the LPA solver. The feasible solution from LPA is

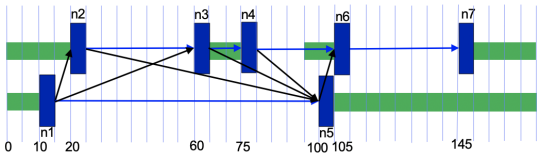


Figure 7: Constraint graph with initial node positions. The location of the nodes n_i is labelled with respect to origin, assuming that origin is at left most metal edge and pitch of the grid is 5 unit and given that W=5, D_t =25, D_u =15.

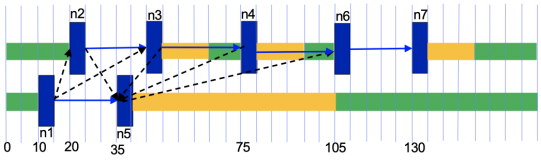


Figure 8: The initial solution is computed with Type "B" arcs (blue), while Type "C" arcs (dotted) are relaxed. Total computed extent wire (orange segments) length =110. Note that arc Type "A" is subsumed by Type "B".

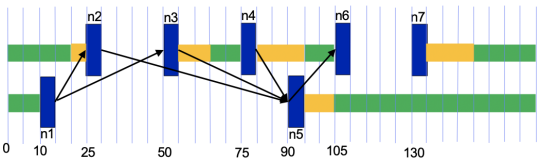


Figure 9: After obtaining initial solution (Figure 8) with Type "B" arcs, Type "C" arcs (black) are added into the graph and solution is computed for them. Total computed extent wire length =55.

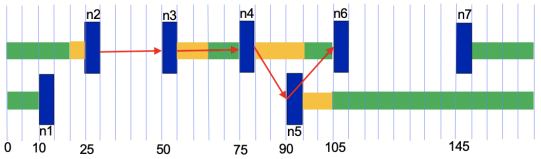


Figure 10: EMA solution on LPA output with node n7 as pivot. Total computed extent wire length =40. A tight arc tree is formed rooted at node n3.

taken as the initial feasible solution. A feasible solution is maintained throughout the algorithm. A node is selected as a pivot for running optimization. A spanning tree T_i of zero slack arcs rooted at pivot node n_i is formed. The slack of the tree is the minimum slack of the arcs not in the tree. The entire tree will be moved, if the move improves the solution cost, ie if $C_i < 0$ (minimize) for:

$$C_i = w_i \sum d_j, j \in T_i.$$

where $w_i \in \{-1, 1\}$ depends on the slack of the tree. The value d_j is the movement of the node n_i based on the slack of the least

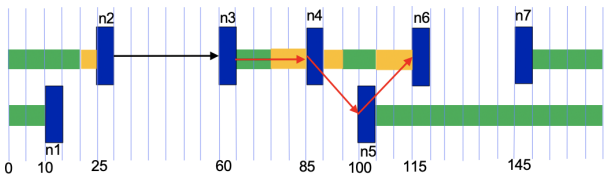


Figure 11: Further improvement with EMA solution selecting node n3 as pivot. Total computed extent wire length =30.

slack arc. Figures 10 and 11 depict the improved solution with final computed extent wire length. In Figure 10 a tight arc tree is created by selecting node n3 as pivot. The arc tree is comprised of black and red arcs, where black arcs have non-zero slack and red arcs have zero slack. Figure 11 shows movement of the nodes in the tight arc tree.

4.3 Routing-based line end optimization (rLEO)

In this section, we present a routing-based line end optimization solution for resolving spacing errors which are not feasible to solve using the LEO solver alone. In detail router, the routing path search is adapted to understand the line end spacing constraints, blocking out areas such that the constraints are satisfied. A fix-up strategy is deployed where, several routing iterations are used to address violations while reducing disruption to existing routing. In addition, the LEO solver itself is used to provide the unsolved problems for the router to work on in lieu of using a violation checker which would be expensive in run time. The line end aware routing heuristic is as follows:

(1) Run **LEO** solver and collect unsolved problems from **LEO** solution. The unsolved problems are unsolved subgraphs, so-called violation graph containing metal shapes in violations. The violation graph has 2 important roles:
   - To enable selection of metal shapes based on their violation type inferred from unsolved connected arcs. This helps in developing violation driven routing strategies for each iteration.
   - For rip up the selection of neighboring shapes is controlled in each iteration and shapes are selected based on connected arc types.

(2) Collect violated nets that are associated with the metal shapes.
(3) Rip up neighbor shapes, run **LEO** solver and route ripped nets. Removing bystander shapes provides room for the LP algorithm to potentially address the original error.
(4) Rip up the error shape and a larger number of neighbor shapes and route with cut layers blocked during the path search. A search window is selected to choose neighboring shapes and the size of the search window is increased in each iteration.

Our results show that this strategy is quite effective in solving a majority of the infeasible line end spacing violations. However, the effectiveness of the overall rLEO solution is limited by local detail routing congestion and the runtime spend in each iterative operation.

5 ROUTING PLUGIN BASED DESIGN FLOW

In this section, we present a new design methodology flow based on our routing plugin. Fig. 12 displays the proposed methodology flow. We start with the placement output and run clock routing as first step. To note that, none of our existing routing tools (clock and signal) are aware of new ground rules related to line end spacing. We use our plugin infrastructure which is comprised of LEO solver and rLEO solver. The plugin is run wherever there is a change in the routing data in the flow for example at the end of clock routing and signal routing. As rLEO solver involves costly rip-up and reroutes, so we run it only at the end of flow after "routing based optimization" step. The "routing based optimization" step involves timing fixup and power recovery along with cleaning up of DRVs.

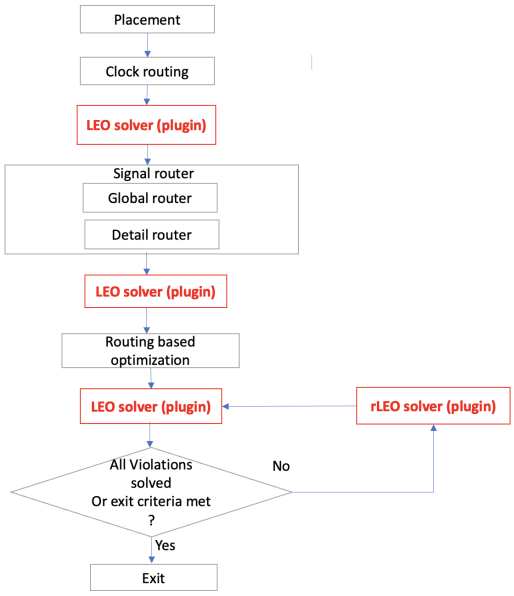


Figure 12: Routing plugin based design flow

6 EXPERIMENTAL RESULTS

Both of the solutions, LEO and rLEO are implemented in C++. All of the experiments are conducted on a Linux machine with a 2.7 GHz CPU and 126 GB memory. The experiments are run on real industry designs, where we selected unique 6 designs for both 7nm EUV and SADP technology. We have taken large designs for our EUV benchmark as shown in Table 2. The graph detail contains the count of nodes, arcs, and metal shapes on the EUV layer. The nodes and arcs are computed from the graph using a pruning factor of 3. We used an industry-standard detail router to get a routing solution for our designs. However, the detail router is not aware of technology-specific line-end rules and produces a solution with thousands of DRVs. We take this data as an input and run our routing plugin on top of it. Our experiments show that our stand-alone routing solution is very effective in solving the remaining DRVs.
We used SADP designs for quick quality and run-time evaluation of the LEO solver and compared it with the ILP solution from the CPLEX solver. This analysis was important to decide early on whether to invest in a "native" solution, and the runtime data in Table 1 motivated us to invest in our own solver. Comparison between columns 7 and 8 shows that LEO solver runs much faster than a CPLEX based solution. As both of them solved all the DRVs, hence the data is not specified in the table. We used "extent length" as another metric to measure the quality of the solution. We found that LEO solver produces a solution with a marginal increase of average 4% in extent length as shown in column 6. This difference arises due to use of heuristic integer variable assignment in LPA solver. For remaining experiments we didn't use any of the SADP designs, as they were very small for the various assessments.

The impact of a prune value on runtime and quality of routing is evaluated in Table 3. We picked up the largest design (with 1.7 million nets and roughly 1.7 million gates) from our EUV benchmark (Design 6) and performed 6 experiments with different prune factors (very low value 1 to a high value 10) as shown in column 1. Column 2 represents the count of subgraphs created. By increasing the prune factor value, nodes in a graph can travel to a much longer distance. As a result, we get fewer subgraphs but with larger sizes. In Columns 3 and 4 we compare the remaining unsolved subgraphs. The runtime data is shown in column 6. With the increase in prune factor, the run time also increases since with larger movement of the nodes, subgraph size increases. The quality of routing is shown with the DRVs in column 7. For prune factor 1, we get a high DRV count, as many of the subgraphs are unsolved due to restricted node movements. Increasing the prune factor shows improvement in the DRVs. Our assessment shows that a pruning factor of 3 holds good for both runtime and DRVs.
In Table 4, we evaluate the effectiveness of the LEO solver based on two metrics. The first metric shows the percentage of unsolved subgraphs. As the LEO solver operates on a detail routing solution that is not line-end aware, we typically get more than a hundred thousand line-end violations in every design to being with. Our data shows that the LEO solver is highly efficient in finding a solution for subgraphs and in all of the cases we examined, always finds a solution if one exists. We get very few unsolved subgraphs and all these are infeasible ones. Compared with the previous work [9], which is focused only on trim mask rules and solves 75% of parity violations, the results show that our algorithm is more efficient and it solves all the line-end (that includes the class of parity violations) violations. Moreover, our experiments are run on very large designs compare to this work which has the largest design with 15,000 nets. The second metric shows the time spent by the solver. For our largest design, the LEO solver takes less than 7 minutes to compute the solution. The previous work in [8], proposes a graph-based SAMP router to do cut mask optimization in self-aligned multiple patterning (SAMP) technology. The run time for running graph-based SAMP router in their largest design with 28,000 nets is close to 1 minute. The runtime for LEO solver for a comparable design is just 12 seconds. Our runtime data in column 5 shows that our solution scales really well with the increase in the design size. Table 5 shows, columns (2 and 3) with initial wire-length and final wire-length on Mx layers. The final wire length is computed after running the LEO solver. Our experiments show that the increase in wire length is less than 2%. In our designs, the overhead of 2% wire-length showed no impact on the wiring congestion and the electrical characteristics,

like timing and delay. The added extent metal shapes are floating wires and do not have an impact on the overall wire resistance. The quality of the solution of the LEO solver is checked by running a design rule checker. The DRVs occur on unsolved subgraphs, and those can't be solved alone by the LEO solver. Therefore we run rLEO solver on unsolved subgraphs and our results show (columns 6 and 7) that it reduces the remaining DRVs by a factor of 10x on average with a marginal runtime cost. Also to note that we run LEO solver in multiple places in our design flow, while rLEO solver is only run at the very end. Therefore the run time penalty due to rip-up and reroute operation is not high.

Table 1: Extent(Ext) length(len) and run time comparison between CPLEX solver and LEO solver.

	Benchmarks		Ext len[mm]		Ext len	runtime[s]	
Designs	#Nets	#Gates	CPLEX	LEO	inc%	CPLEX	LEO
SADP1	3680	8971	0.51	0.58	1.27	28	1.2
SADP2	9743	25948	1.39	1.45	4.31	101	4.8
SADP3	14278	45903	1.90	1.98	4.21	33	4.7
SADP4	19718	81547	2.13	2.20	3.28	50	4.5
SADP5	26754	85974	3.81	3.98	4.46	361	9.0
SADP6	30600	67573	4.97	5.20	4.62	1262	12.0

Table 2: Description of test cases.

	EUV Benchmarks		Graph details		
Designs	# Nets	# Gates	# Nodes	# Arcs	# shapes
EUV1	234,621	210,874	6,712,374	2,918,169	3,311,686
EUV2	244,220	225,659	6,026,936	3,416,334	2,999,981
EUV3	402,688	361,437	15,413,760	5,268,430	7,697,268
EUV4	720,490	611,223	16,740,799	8,432,198	8,333,372
EUV5	861,422	793,193	19,565,996	10,906,504	9,775,062
EUV6	1,738,299	1,746,638	46,601,419	23,116,490	23,243,333

Table 3: Impact of pruning the position of any line end edge factor (P.F) on finding subgraph (S.G.) solution and total execution time. Evaluated on design EUV6.

P.F	Total sub-graphs	S.G. Unsolved	S.G. Unsolved%	Run-time (sec)	#DRV
1	31,678,471	636,295	2.008	360	46,000
2	27,259,736	17,973	0.065	380	260
3	25,394,026	6,383	0.025	390	245
4	23,520,178	4,932	0.020	380	245
5	22,172,952	4,924	0.022	480	245
10	17,763,543	4,889	0.027	6600	245

Table 4: Evaluation of LEO solver for finding subgraph solution and total execution time.

Designs	Total sub-graphs	Sub-graph unsolved	Sub-graph unsolved%	Run-time (sec)
EUV1	4,233,742	42	0.0010	38
EUV2	3,156,474	9	0.0003	40
EUV3	11,063,076	186	0.0017	120
EUV4	8,741,186	136	0.0016	130
EUV5	9,534,048	98	0.0010	160
EUV6	25,394,026	226	0.0009	400

Table 5: Evaluation of LEO solver for extent length optimization and solution quality.

Designs	Initial WL[mm]	Final WL[mm]	Incresed Len%	LEO #DRV	rLEO #DRV	rLEO runtime(s)
EUV1	3,385.19	3,439.59	1.61	48	5	180
EUV2	3,666.34	3,725.84	1.62	20	0	120
EUV3	7,429.12	7,534.63	1.42	100	15	360
EUV4	10,185.17	10,358.17	1.70	180	45	700
EUV5	12,587.20	12,837.39	1.99	150	25	650
EUV6	26,829.64	27,345.69	1.92	245	60	1200

7 CONCLUSIONS

In this paper, we proposed a routing plugin for 7nm EUV and SADP technologies. We derived a common mathematical formulation for both of the technologies that helped us to create a common infrastructure for optimization. We modeled the problem using a constraint graph and a new graph pruning technique was demonstrated, which proved to be very effective in solving multiple subgraphs independently. A set of algorithms were introduced to solve the constraint graph. We proposed rLEO solution where we integrated line end optimizer and detail router to improve routing quality. In the end we introduced a novel design methodology flow based on our plugin. During the development of our 7nm based microprocessor, the plugin allowed us to work on different technologies without making any change in existing routing tools (clock and signal). This enabled a very fast design convergence cycle. Additionally, our experimental data showed that our solver always produces a legal solution if one exists. In comparison to the closest work [9], our quality is 25% better and we solve additional class of violations as well. Additionally, The LEO solver operates in small area to fix the violations, it showed negligible impact on local congestion and timing in our 7nm designs.

REFERENCES

[1] L. Liebmann, L. Pileggi, J. Hibbeler, V. Rovner, T. Jhaveri, and G. Northrop, "Simplify to survive: prescriptive layouts ensure profitable scaling to 32nm and beyond," in *Design for Manufacturability through Design-Process Integration III*, vol. 7275, p. 72750A, International Society for Optics and Photonics, 2009.

[2] H. Zhang, Y. Du, M. D. Wong, and K.-Y. Chao, "Mask cost reduction with circuit performance consideration for self-aligned double patterning," in *Design Automation Conference (ASP-DAC), 2011 16th Asia and South Pacific*, pp. 787–792, IEEE, 2011.

[3] H. Zhang, Y. Du, M. D. Wong, and R. Topaloglu, "Self-aligned double patterning decomposition for overlay minimization and hot spot detection," in *Proceedings of the 48th Design Automation Conference*, pp. 71–76, IEEE, 2011.

[4] Y. Ban, K. Lucas, and D. Pan, "Flexible 2d layout decomposition framework for spacer-type double pattering lithography," in *Proceedings of the 48th Design Automation Conference*, pp. 789–794, ACM, 2011.

[5] Z. Xiao, Y. Du, H. Zhang, and M. D. Wong, "A polynomial time exact algorithm for self-aligned double patterning layout decomposition," in *Proceedings of the 2012 ACM international symposium on International Symposium on Physical Design*, pp. 17–24, ACM, 2012.

[6] H. Zhang, M. D. Wong, and K.-Y. Chao, "On process-aware 1-d standard cell design," in *Proceedings of the 2010 Asia and South Pacific Design Automation Conference*, pp. 838–842, IEEE Press, 2010.

[7] X. Xu, B. Cline, G. Yeric, B. Yu, and D. Z. Pan, "Self-aligned double patterning aware pin access and standard cell layout co-optimization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, pp. 699–712, May 2015.

[8] S. Y. Fang, "Cut mask optimization with wire planning in self-aligned multiple patterning full-chip routing," in *The 20th Asia and South Pacific Design Automation Conference*, pp. 396–401, Jan 2015.

[9] Y. Wei, R. Samanta, and Y. Li, "Lesar: A dynamic line-end spacing aware detailed router," in *2018 Design, Automation Test in Europe Conference Exhibition (DATE)*, pp. 1473–1476, March 2018.

[10] P. Cremer, S. Hougardy, J. Schneider, and J. Silvanus, "Automatic cell layout in the 7nm era," in *Proceedings of the 2017 ACM on International Symposium on Physical Design*, pp. 99–106, ACM, 2017.